Rodriguez Navarrete angel Gabriel

Semana12

Gestión de inventarios una empresa de comercio electrónico necesita implementar un sistemas de gestión de inventarios para controlar la cantidad de productos disponibles en un almacen. El sistema debe permitir registrar productos, actualizar cantidades, eliminar productos y realizar consultas.

1- Diseñar una estrcutura de datos para almacenar la información de los productos (código,nombre,cantidad,precio)
2- Implementar operaciones para registrar un nuevo producto ,actualizar la cantidad de un producto existente, eliminar un producto, consultar la cantidad de un producto en especifico y consultar todos los productos con una cantidad mayor a un valor en especifico
3- Considerar la eficiencia del uso de memoria y la velocidad de acceso a los datos

Ejercicios y preguntas
Cual es la estructura de datos mas adecuada para este problema (árbol,lista,tabla,hash,otro menciar cual) justificar su respuesta

Diseña la estructura de datos selecionada y explica los puntos que la hacen mas edecuada

Implementar las operaciones requeridas utilizando la estructura de datos diseñada

Cual es el costo temporal o parcial de cada operación

Como se podría mejorar la eficiencia del sistema

Que suscede si se intenta registrar un producto con un código existente

```
# -*- coding: utf-8 -*-
import tkinter as tk
from tkinter import messagebox
import json
import os

class Inventario:
    def __init__(self, archivo='inventario.json'):
        self.productos = {}
        self.archivo = archivo
        self.cargar_datos()

    def registrar_producto(self, codigo, nombre, cantidad, precio):
        if codigo in self.productos:
            raise ValueError("El producto con este código ya existe.")
        self.productos[codigo] = {
```

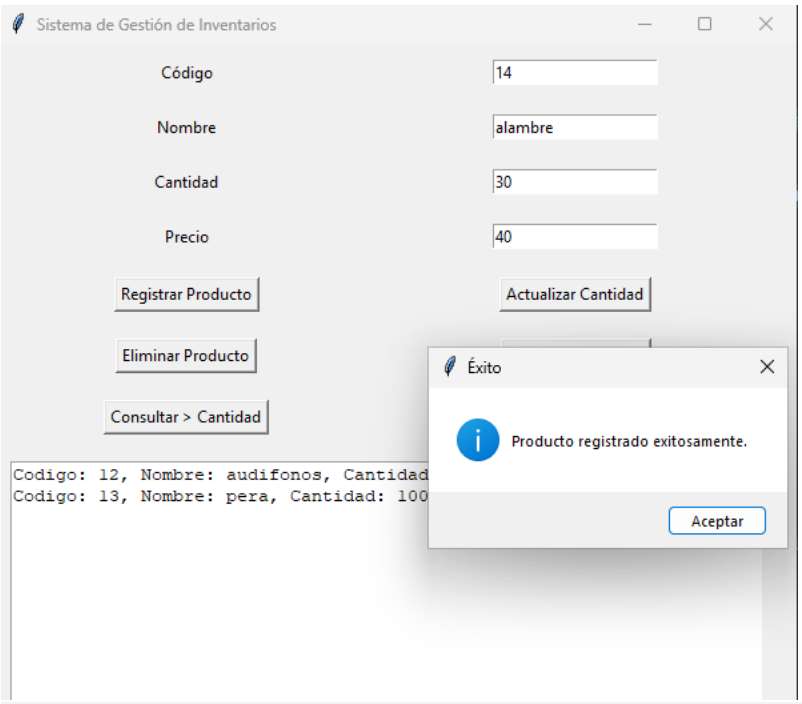
```
            'nombre': nombre,
            'cantidad': cantidad,
            'precio': precio
        }
        self.guardar_datos()

    def actualizar_cantidad(self, codigo, nueva_cantidad):
        if codigo not in self.productos:
            raise KeyError("El producto no existe.")
        self.productos[codigo]['cantidad'] = nueva_cantidad
        self.guardar_datos()

    def eliminar_producto(self, codigo):
        if codigo not in self.productos:
            raise KeyError("El producto no existe.")
        del self.productos[codigo]
        self.guardar_datos()

    def consultar_cantidad(self, codigo):
        if codigo not in self.productos:
            raise KeyError("El producto no existe.")
        return self.productos[codigo]['cantidad']

    def consultar_productos_mayor_cantidad(self, cantidad_minima):
        return {codigo: info for codigo, info in self.productos.items() if
info['cantidad'] > cantidad_minima}

    def obtener_todos_productos(self):
        return self.productos

    def guardar_datos(self):
        with open(self.archivo, 'w') as f:
            json.dump(self.productos, f, indent=4)

    def cargar_datos(self):
        if os.path.exists(self.archivo):
            with open(self.archivo, 'r') as f:
                self.productos = json.load(f)

class Aplicacion(tk.Tk):
    def __init__(self):
        super().__init__()
        self.inventario = Inventario()
        self.title("Sistema de Gestión de Inventarios")
        self.geometry("600x500")
```

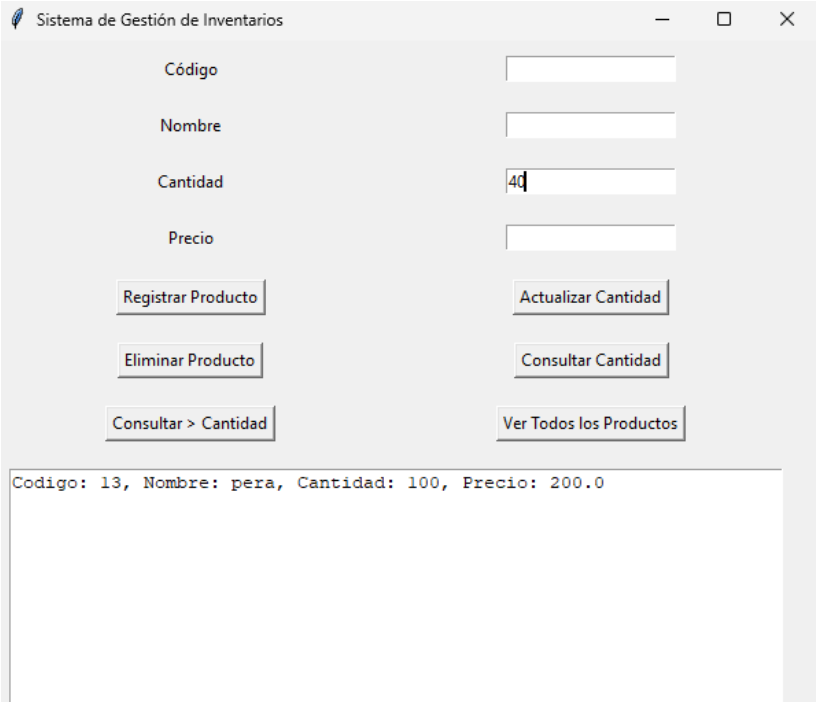
```
        # Widgets
        self.create_widgets()

    def create_widgets(self):
        # Labels and Entries
        tk.Label(self, text="Código").grid(row=0, column=0, padx=10,
pady=10)
        tk.Label(self, text="Nombre").grid(row=1, column=0, padx=10,
pady=10)
        tk.Label(self, text="Cantidad").grid(row=2, column=0, padx=10,
pady=10)
        tk.Label(self, text="Precio").grid(row=3, column=0, padx=10,
pady=10)
        self.codigo_entry = tk.Entry(self)
        self.nombre_entry = tk.Entry(self)
        self.cantidad_entry = tk.Entry(self)
        self.precio_entry = tk.Entry(self)
        self.codigo_entry.grid(row=0, column=1, padx=10, pady=10)
        self.nombre_entry.grid(row=1, column=1, padx=10, pady=10)
        self.cantidad_entry.grid(row=2, column=1, padx=10, pady=10)
        self.precio_entry.grid(row=3, column=1, padx=10, pady=10)
        # Buttons
        tk.Button(self, text="Registrar Producto",
command=self.registrar_producto).grid(row=4, column=0, padx=10, pady=10)
        tk.Button(self, text="Actualizar Cantidad",
command=self.actualizar_cantidad).grid(row=4, column=1, padx=10, pady=10)
        tk.Button(self, text="Eliminar Producto",
command=self.eliminar_producto).grid(row=5, column=0, padx=10, pady=10)
        tk.Button(self, text="Consultar Cantidad",
command=self.consultar_cantidad).grid(row=5, column=1, padx=10, pady=10)
        tk.Button(self, text="Consultar > Cantidad",
command=self.consultar_productos_mayor_cantidad).grid(row=6, column=0,
padx=10, pady=10)
        tk.Button(self, text="Ver Todos los Productos",
command=self.ver_todos_productos).grid(row=6, column=1, padx=10, pady=10)
        # Text widget for output
        self.output_text = tk.Text(self, height=15, width=70)
        self.output_text.grid(row=7, column=0, columnspan=2, padx=10,
pady=10)

    def registrar_producto(self):
        try:
            codigo = self.codigo_entry.get()
            nombre = self.nombre_entry.get()
            cantidad = int(self.cantidad_entry.get())
```

```
            precio = float(self.precio_entry.get())
            self.inventario.registrar_producto(codigo, nombre, cantidad,
precio)
            messagebox.showinfo("Éxito", "Producto registrado
exitosamente.")
            self.limpiar_entradas()
        except ValueError as e:
            messagebox.showerror("Error", str(e))

    def actualizar_cantidad(self):
        try:
            codigo = self.codigo_entry.get()
            nueva_cantidad = int(self.cantidad_entry.get())
            self.inventario.actualizar_cantidad(codigo, nueva_cantidad)
            messagebox.showinfo("Éxito", "Cantidad actualizada
exitosamente.")
            self.limpiar_entradas()
        except KeyError as e:
            messagebox.showerror("Error", str(e))
        except ValueError:
            messagebox.showerror("Error", "Cantidad debe ser un número
entero.")

    def eliminar_producto(self):
        try:
            codigo = self.codigo_entry.get()
            self.inventario.eliminar_producto(codigo)
            messagebox.showinfo("Éxito", "Producto eliminado exitosamente.")
            self.limpiar_entradas()
        except KeyError as e:
            messagebox.showerror("Error", str(e))

    def consultar_cantidad(self):
        try:
            codigo = self.codigo_entry.get()
            cantidad = self.inventario.consultar_cantidad(codigo)
            self.output_text.delete(1.0, tk.END)
            self.output_text.insert(tk.END, f"Cantidad de producto
'{codigo}': {cantidad}\n")
        except KeyError as e:
            messagebox.showerror("Error", str(e))

    def consultar_productos_mayor_cantidad(self):
        try:
            cantidad_minima = int(self.cantidad_entry.get())
```

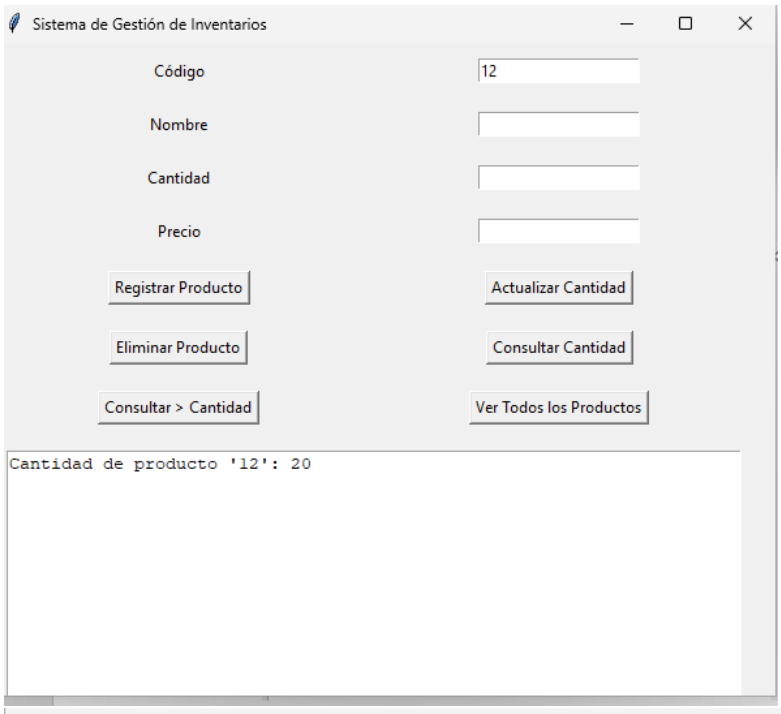
```
            productos =
self.inventario.consultar_productos_mayor_cantidad(cantidad_minima)
            self.output_text.delete(1.0, tk.END)
            if productos:
                for codigo, info in productos.items():
                    self.output_text.insert(tk.END, f"Codigo: {codigo},
Nombre: {info['nombre']}, Cantidad: {info['cantidad']}, Precio:
{info['precio']}\n")
            else:
                self.output_text.insert(tk.END, "No hay productos con
cantidad mayor a la especificada.\n")
        except ValueError:
            messagebox.showerror("Error", "Cantidad mínima debe ser un
número entero.")

    def ver_todos_productos(self):
        productos = self.inventario.obtener_todos_productos()
        self.output_text.delete(1.0, tk.END)
        if productos:
            for codigo, info in productos.items():
                self.output_text.insert(tk.END, f"Codigo: {codigo}, Nombre:
{info['nombre']}, Cantidad: {info['cantidad']}, Precio: {info['precio']}\n")
        else:
            self.output_text.insert(tk.END, "No hay productos en el
inventario.\n")

    def limpiar_entradas(self):
        self.codigo_entry.delete(0, tk.END)
        self.nombre_entry.delete(0, tk.END)
        self.cantidad_entry.delete(0, tk.END)
        self.precio_entry.delete(0, tk.END)

if __name__ == "__main__":
    app = Aplicacion()
    app.mainloop()
```

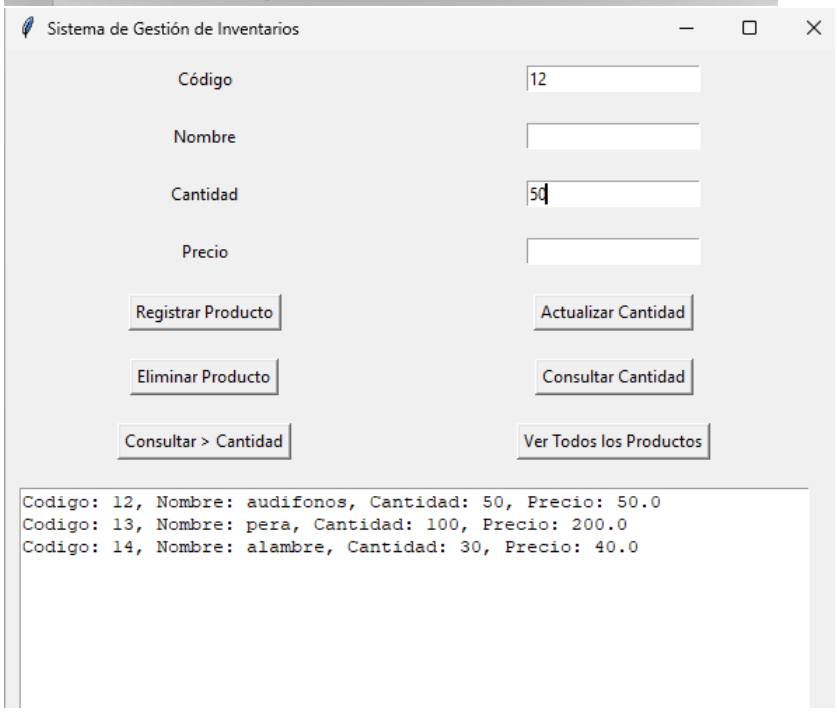
Rodriguez Navarrete angel Gabriel



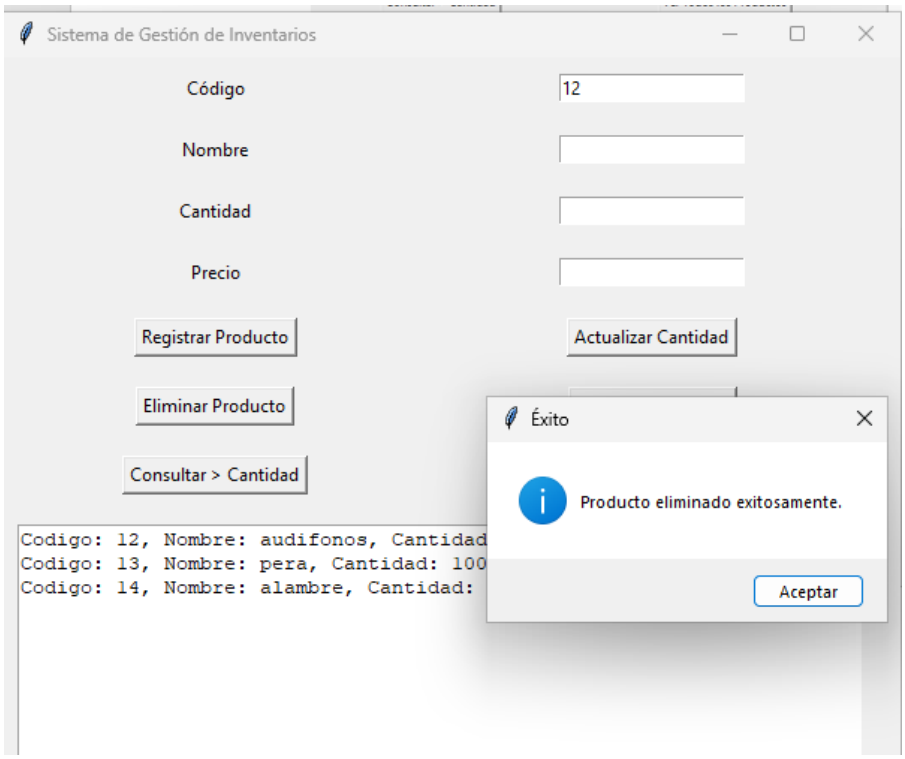


Rodriguez Navarrete angel Gabriel





Rodriguez Navarrete angel Gabriel



1. ¿Cuál es la estructura de datos más adecuada para este problema (árbol, lista, tabla, hash, otro mencionar cuál)? Justificar su respuesta.

La estructura de datos más adecuada es una tabla hash.

Justificación:

Acceso rápido: La tabla hash permite un acceso rápido a los datos, con operaciones de búsqueda, inserción, actualización y eliminación con un costo promedio de O(1).

Clave única: El código del producto puede ser usado como clave única en la tabla hash, asegurando que no haya duplicados.

Flexibilidad: Es ideal para almacenar datos con claves únicas y valores asociados, como los productos en un inventario.

2. Diseña la estructura de datos seleccionada y explica los puntos que la hacen más adecuada.

Diseño de la Tabla Hash:

Clave: El código del producto (único para cada producto).

Rodriguez Navarrete angel Gabriel

Valor: Un diccionario que contiene los detalles del producto (nombre, cantidad, precio).

Puntos que la hacen adecuada:

Acceso directo: Permite acceder rápidamente a los productos usando su código.

Facilidad de manejo: Agregar, actualizar y eliminar productos es eficiente.

Escalabilidad: Puede manejar grandes cantidades de datos sin una disminución significativa en el rendimiento.

3. Implementar las operaciones requeridas utilizando la estructura de datos diseñada.

Las operaciones se implementan de la siguiente manera en una tabla hash:

Registrar Producto: Insertar el producto en la tabla hash con el código como clave.

Actualizar Cantidad: Buscar el producto por su código y actualizar su cantidad.

Eliminar Producto: Buscar el producto por su código y eliminarlo de la tabla.

Consultar Cantidad: Buscar el producto por su código y devolver su cantidad.

Consultar Productos con Cantidad Mayor a una Especificada: Iterar sobre todos los productos y filtrar los que cumplen la condición.

Obtener Todos los Productos: Devolver todos los productos en la tabla hash.

4. ¿Cuál es el costo temporal o parcial de cada operación?

Registrar Producto: O(1) en promedio.

Actualizar Cantidad: O(1) en promedio.

Eliminar Producto: O(1) en promedio.

Consultar Cantidad: O(1) en promedio.

Consultar Productos con Cantidad Mayor a una Especificada: O(n), donde n es el número de productos.

Obtener Todos los Productos: O(n), donde n es el número de productos.

5. ¿Cómo se podría mejorar la eficiencia del sistema?

Rehashing: Implementar rehashing para evitar que la tabla se llene y mantenga la eficiencia.

Índices secundarios: Crear índices adicionales basados en atributos frecuentemente consultados, como cantidad o precio.

Caching: Utilizar técnicas de caching para acelerar las operaciones de lectura frecuentes.

Rodriguez Navarrete angel Gabriel

Optimización de consultas: Optimizar las consultas que requieren recorrer todos los elementos, como la consulta de productos con cantidad mayor a una especificada.

6. ¿Qué sucede si se intenta registrar un producto con un código existente?

Si se intenta registrar un producto con un código que ya existe:

El sistema debe lanzar una excepción o mostrar un mensaje de error indicando que el producto con ese código ya existe, evitando así la duplicación de códigos en el inventario.

Caso practico numero 2

gestion de una biblioteca

una biblioteca necesita implementar un sistema para gestionar sus libros prestamos y usuarios el sistema

debe permitir registrar libros prestamos y usuarios asi como tambien realizar consultas y actualizaciones

debes realizar lo siguiente diseña una estrructura de datos para almecenar los datos de los

libros(titulo,autor,genero,estado) minmo 50 libros de ingeneria en sistemas

2 diseñar una estructura de datos para almacenar la informacion de los prestamos,usuario,libro,fecha de

prestamo, y fecha de devolucion minimo 10 usuarios

3 diseñar una estructura de datos para almacenar la informacion de los usuarios

(nombre,direccion,telefono)

4implemetnar operaciones para:

+registrar un nuevo libro

+registrar un nuevo prestamo

+registrar un nuevo usuario

+consultar la disponibilidad de un libro

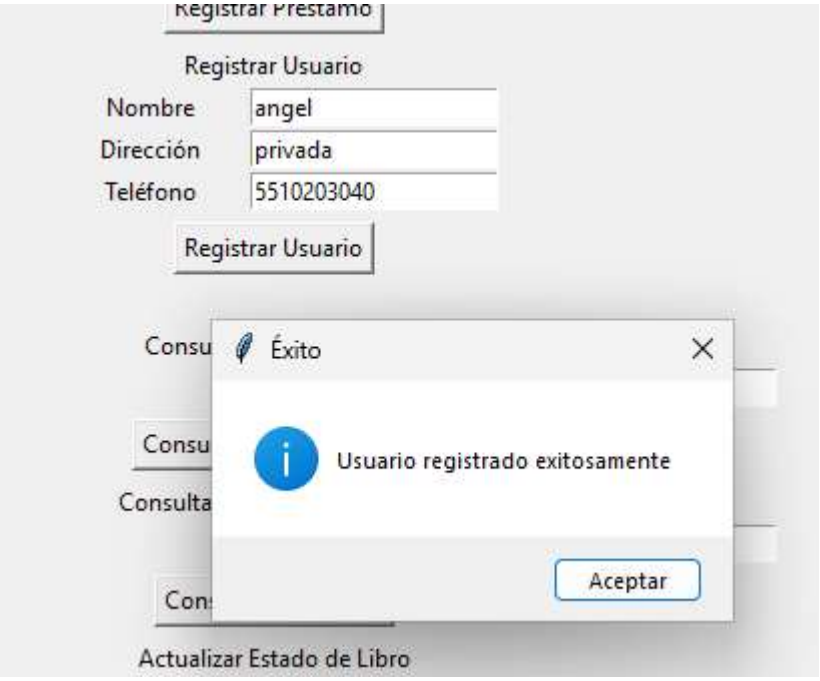
+consultar los prestamos de un libro

+actualizar el estado de un libro

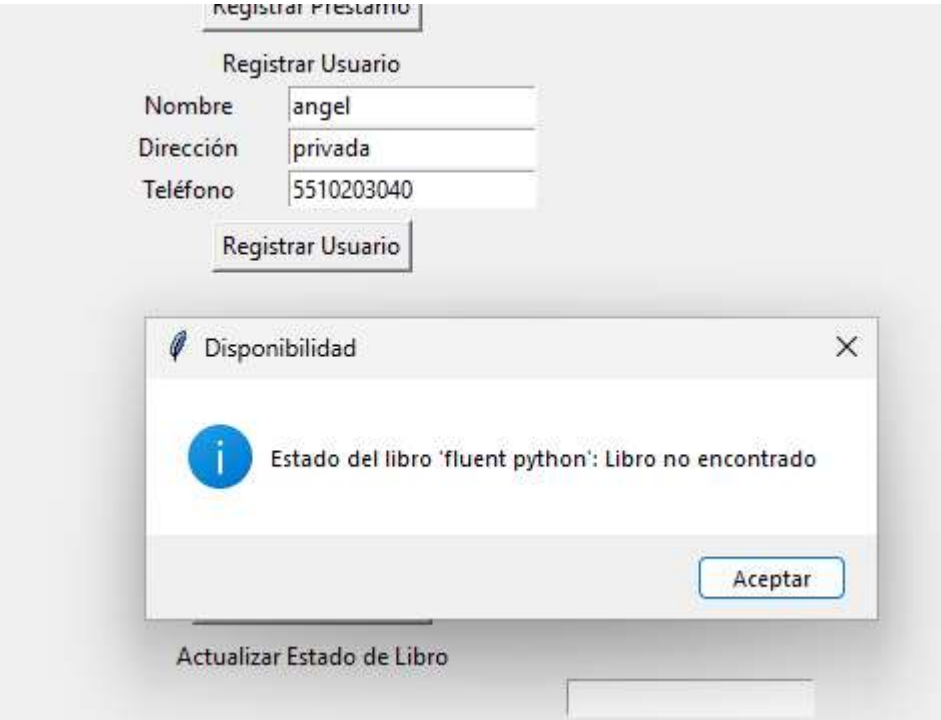
+considerar la eficencia del uso de la memoria y la velocidad de acceso en los datos

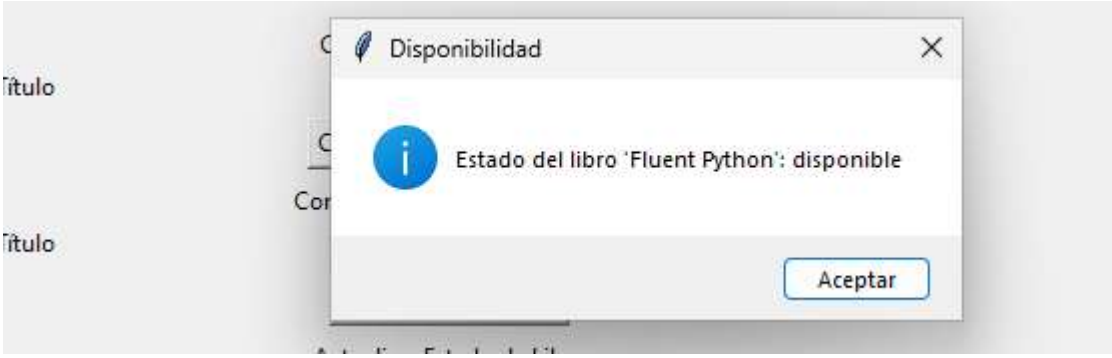
Rodriguez Navarrete angel Gabriel

Registro de usuario

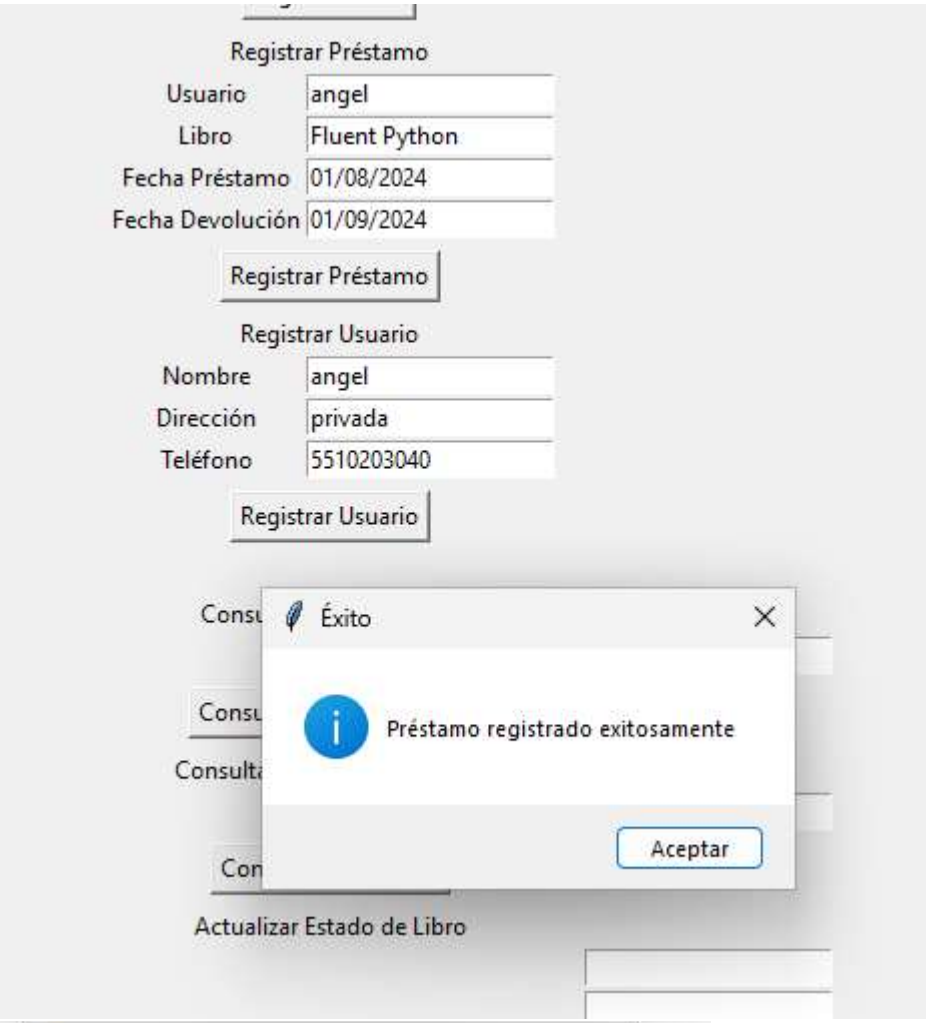


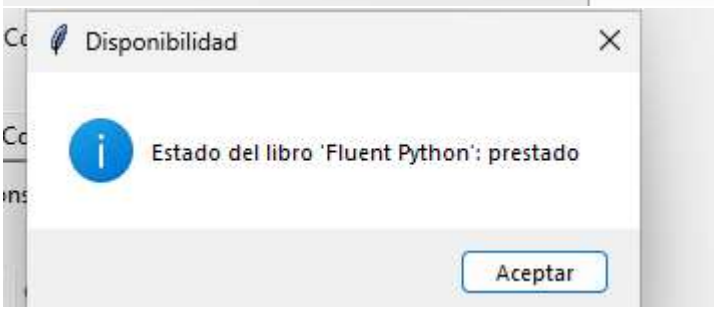
Rodriguez Navarrete angel Gabriel





Rodriguez Navarrete angel Gabriel





Libros

Título	Autor	Género	Estado
Clean Code	Robert C. Martin	Ingeniería en Sistemas	disponible
The Pragmatic Programmer	Andrew Hunt, David Thomas	Ingeniería en Sistemas	disponible
Design Patterns	Erich Gamma, Richard Helm, Ralph J	Ingeniería en Sistemas	disponible
Refactoring	Martin Fowler	Ingeniería en Sistemas	disponible
Code Complete	Steve McConnell	Ingeniería en Sistemas	disponible
Introduction to Algorithms	Thomas H. Cormen, Charles E. Leiser	Ingeniería en Sistemas	disponible
Artificial Intelligence: A Modern App	Stuart Russell, Peter Norvig	Ingeniería en Sistemas	disponible
Structure and Interpretation of Com	Harold Abelson, Gerald Jay Sussman	Ingeniería en Sistemas	disponible
Computer Networks	Andrew S. Tanenbaum, David J. Wetl	Ingeniería en Sistemas	disponible
Operating System Concepts	Abraham Silberschatz, Greg Gagne, F	Ingeniería en Sistemas	disponible

Rodriguez Navarrete angel Gabriel

Structure and Interpretation of Comp	Harold Abelson, Gerald Jay Sussman	Ingeniería en Sistemas	disponible
Computer Networks	Andrew S. Tanenbaum, David J. Wetl	Ingeniería en Sistemas	disponible
Operating System Concepts	Abraham Silberschatz, Greg Gagne, F	Ingeniería en Sistemas	disponible
The Art of Computer Programming	Donald E. Knuth	Ingeniería en Sistemas	disponible
Database System Concepts	Abraham Silberschatz, Henry F. Kortl	Ingeniería en Sistemas	disponible
Introduction to the Theory of Compu	Michael Sipser	Ingeniería en Sistemas	disponible
Computer Organization and Design	David A. Patterson, John L. Hennessy	Ingeniería en Sistemas	disponible
Software Engineering	Ian Sommerville	Ingeniería en Sistemas	disponible
Compilers: Principles, Techniques, ar	Alfred V. Aho, Monica S. Lam, Ravi S	Ingeniería en Sistemas	disponible
Introduction to Automata Theory, La	John E. Hopcroft, Rajeev Motwani, Je	Ingeniería en Sistemas	disponible

Título	Autor	Género	Estado
Distributed Systems: Principles and F	Andrew S. Tanenbaum, Maarten Van	Ingeniería en Sistemas	disponible
Software Architecture in Practice	Len Bass, Paul Clements, Rick Kazma	Ingeniería en Sistemas	disponible
Computer Systems: A Programmer's	Randal E. Bryant, David R. O'Hallaron	Ingeniería en Sistemas	disponible
Computer Architecture: A Quantitati	John L. Hennessy, David A. Patterson	Ingeniería en Sistemas	disponible
Modern Operating Systems	Andrew S. Tanenbaum, Herbert Bos	Ingeniería en Sistemas	disponible
Algorithm Design Manual	Steven S. Skiena	Ingeniería en Sistemas	disponible
Spring in Action	Craig Walls	Ingeniería en Sistemas	disponible
Effective Java	Joshua Bloch	Ingeniería en Sistemas	disponible
Java Concurrency in Practice	Brian Goetz	Ingeniería en Sistemas	disponible
Python Crash Course	Eric Matthes	Ingeniería en Sistemas	disponible

Título	Autor	Género	Estado
The Phoenix Project	Gene Kim, Kevin Behr, George Spaffc	Ingeniería en Sistemas	disponible
The DevOps Handbook	Gene Kim, Patrick Debois, John Willis	Ingeniería en Sistemas	disponible
You Don't Know JS	Kyle Simpson	Ingeniería en Sistemas	disponible
Eloquent JavaScript	Marijn Haverbeke	Ingeniería en Sistemas	disponible
JavaScript: The Good Parts	Douglas Crockford	Ingeniería en Sistemas	disponible
Learn Python the Hard Way	Zed A. Shaw	Ingeniería en Sistemas	disponible
Programming Rust	Jim Blandy, Jason Orendorff	Ingeniería en Sistemas	disponible
The Rust Programming Language	Steve Klabnik, Carol Nichols	Ingeniería en Sistemas	disponible
Effective C++	Scott Meyers	Ingeniería en Sistemas	disponible
The C++ Programming Language	Bjarne Stroustrup	Ingeniería en Sistemas	disponible